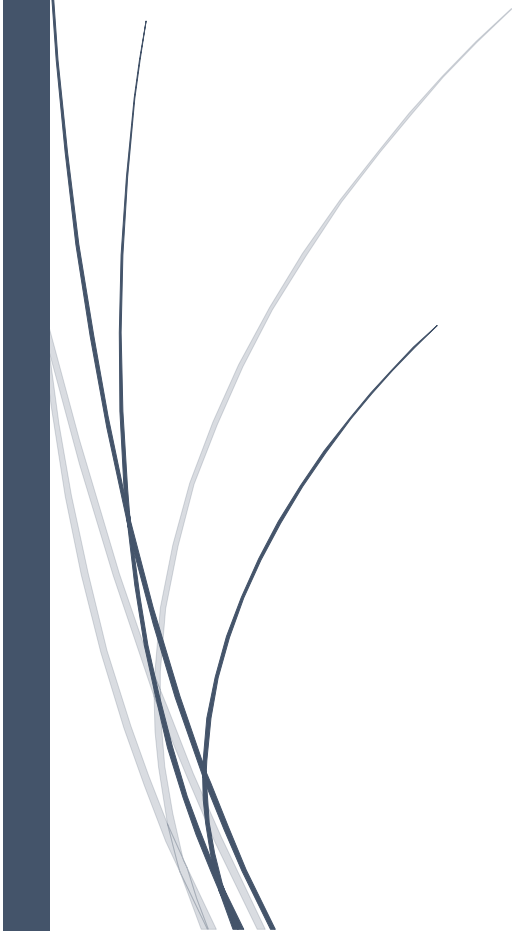
@VANZA

UT4. POO en Python

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. 2
2. Clase....2
3. Atributos....2
3.1. Atributos de instancia....3
3.2. Atributos de clase....3
4. Inicializar clases....4
5. Operar con los atributos....5
6. Privacidad....7
7. Construcción de clases personalizadas....8
8. Propiedades en Clases....11
9. Métodos....15
9.2. Métodos de instancia. 15
9.3. Métodos de clase. 16
9.4. Métodos estáticos. 18
10. Métodos mágicos. 18
11. Personalizar el acceso a los atributos. 21
12. Polimorfismo (Duck Typing)....22
13. Herencia. 23
13.1. Herencia Múltiple. 25
13.2. La clases MIXIN....27
14. Conclusión. 30

1. Introducción.

Como deberíamos saber a estas alturas del ciclo, la POO (programación orientada a objetos) es un paradigma de programación que organiza el código mediante **objetos.** Éstos son representaciones de entidades del mundo real o conceptual, combinando sus propiedades (**atributos**) y comportamientos (**métodos**) en una estructura que facilita el diseño, la reutilización y el mantenimiento del código. Esto va a permitir modelar problemas complejos en componentes más pequeños y reutilizables.

Todos los datos en Python son objetos, las funciones, las estructuras de datos e incluso los literales.

Cualquier objeto se define como una instancia de una **clase.** Una **clase** es una plantilla o modelo que define las propiedades y comportamientos de un objeto(o muchos).

Todo esto lo entenderemos mejor a medida que profundicemos en la unidad y veamos ejemplos concretos.

2. Clase

Para definir una clase se utiliza la palabra reservada **class** seguida de un identificador con nomenclatura CamelCase y **:**

```
class MiClase:
    pass
```

El ejemplo anterior es la definición de clase más simple que se puede hacer, dado que se trata de una clase vacía.

3. Atributos

Los atributos son variables asociadas a una clase u objeto que almacenan datos o información sobre las instancias.

En Python, los atributos se dividen en dos tipos:

3.1. Atributos de instancia.

Son propios de cada objeto(instancia) que se crea a partir de una clase. Se definen dentro del constructor (__init__) o en otros métodos de la clase. Cada instancia tiene su propia copia de los atributos y su valor puede variar para cada objeto.

Ejemplo:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre  # Atributo de instancia
        self.edad = edad      # Atributo de instancia


# Crear dos objetos de la clase Persona
persona1 = Persona("Ana", 30)
persona2 = Persona("Luis", 25)

# Los atributos son únicos para cada instancia
print(persona1.nombre, persona1.edad)  # Salida: Ana 30
print(persona2.nombre, persona2.edad)  # Salida: Luis 25
```

3.2. Atributos de clase

Estos atributos son compartidos por todas las instancias de la clase. Se definen directamente en la clase, fuera de los métodos. Todas las instancias acceden al mismo valor del atributo, salvo que sea sobrescrito en una instancia específica.

Ejemplo:

```
class Vehiculo:
    ruedas = 4  # Atributo de clase (compartido por todas las instancias)

# Crear dos objetos de la clase Vehiculo
vehiculo1 = Vehiculo()
vehiculo2 = Vehiculo()

# Acceso al atributo de clase
print(vehiculo1.ruedas)  # Salida: 4
print(vehiculo2.ruedas)  # Salida: 4

# Modificar el atributo de clase desde la clase
Vehiculo.ruedas = 6
print(vehiculo1.ruedas)  # Salida: 6
print(vehiculo2.ruedas)  # Salida: 6
```

```
#sobrescribimos el valor del atributo ruedas para el vehiculo1
vehiculo1.ruedas=12
print(vehiculo1.ruedas)  # Salida: 12
print(vehiculo2.ruedas)  # Salida: 6
```

4. Inicializar clases

Cuando instanciamos un objeto en Python, se ejecutan varios métodos predefinidos. Uno de ellos es __**init**__ que es el método de inicialización (constructor). Siempre tiene como primer parámetro la propia instancia que se quiere inicializar que, por convención, se denomina **self.** Además de este parámetro, puede definir tantos parámetros como sean necesarios para la inicialización de una instancia. Su uso fundamental es para inicializar los atributos de instancia, aunque se puede utilizar para desarrollar cualquier lógica de programa.

Ejemplo:

```
class Coche:
    def __init__(self, marca, modelo, anio):
        self.marca = marca
        self.modelo = modelo
        self.anio = anio

    def arrancar(self):
        print(f"El {self.marca} {self.modelo} del {self.anio} está arrancando.")

    def detener(self):
        print(f"El {self.marca} {self.modelo} del {self.anio} se ha detenido.")

# Crear instancias de la clase Coche
coche1 = Coche("Toyota", "Corolla", 2020)
coche2 = Coche("Ford", "Focus", 2018)

# Usar los métodos de la clase
coche1.arrancar()  # Salida: El Toyota Corolla del 2020 está arrancando.
coche2.detener()   # Salida: El Ford Focus del 2018 se ha detenido.
```



Modifica el ejemplo anterior para añadir un nuevo atributo de instancia que recoja la aceleración de cualquier coche y crea una función que muestre la aceleración del coche. Por ejemplo:

Salida: El Toyota Corolla acelera de 0-100 Km/h en 10 segundos.

5. Operar con los atributos

La forma más usual de acceder a los atributos es mediante el carácter ".". Pero además, Python proporciona las siguientes funciones para trabajar con los atributos.

- **getattr(obj, nombre[, valor_por_defecto])**→ devuelve el valor del atributo **nombre** del objeto **obj** pasado como parámetro. El **nombre** es una cadena de caracteres y si no existe, devuelve el **valor_por_defecto**, si se ha pasado como parámetro (de lo contrario se eleva una excepción **AttributeError.**
- **setattr(obj, nombre, valor)**→ permite asignar el **valor** al atributo definido como **nombre** (debe ser una cadena) del objeto definido como **obj.** Si el atributo no existe se añade a la instancia y si existe simplemente se actualiza su valor.
- **hasattr(obj, nombre)**→ comprueba si existe el atributo **nombre (**debe ser una cadena) en el objeto **obj,** devolviendo **True** si existe o **False** si no existe.
- **delattr(obj, nombre)**→ elimina el atributo **nombre** (debe ser una cadena) del objeto **obj.**

El siguiente ejemplo es una muestra de todas las funciones anteriores:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

persona = Persona("Juan", 30)

# Obtener el valor de un atributo existente
print(getattr(persona, "nombre"))  # Juan

# Intentar obtener un atributo inexistente con valor por defecto
print(getattr(persona, "direccion", "Desconocida"))  # Desconocida




# Si no se proporciona un valor por defecto, genera AttributeError
try:
    print(getattr(persona, "direccion"))
except AttributeError as e:
    print(e)  # 'Persona' object has no attribute 'direccion'

# Crear un nuevo atributo
setattr(persona, "apellidos", "Gómez Martín")
print(persona.apellidos)  # Gómez Martín
```

```
# Actualizar un atributo existente
setattr(persona, "nombre", "Laura")
print(persona.nombre)  # Laura en lugar de Juan

# Verificar atributos existentes
print(hasattr(persona, "edad"))  # True

# Verificar un atributo inexistente
print(hasattr(persona, "direccion"))  # False

# Eliminar un atributo existente
delattr(persona, "nombre")

# Intentar acceder al atributo después de eliminarlo genera un error
try:
    print(persona.nombre)
except AttributeError as e:
    print(e)  # 'Persona' object has no attribute 'nombre'
```



Debemos prestar atención cuando usamos objetos mutables como valores de atributos de clase. Al estar en el contexto de la clase y no de una instancia, si se modifica el valor del atributo de clase se va a modificar para todas las instancias de la misma clase.

Ejemplo:

```
class Gato:
    num_patas = 4
    orejas = 2
    nombres = []
    def __init__(self, nombre):
        self.nombre = nombre
        self.nombres.append(nombre)

garfield = Gato('Garfield')
bigotes = Gato('Bigotes')

print(Gato.nombres) # ['Garfield','Bigotes']
print(bigotes.nombres) # ['Garfield','Bigotes']

#Observa que ahora modificamos el atributo nombres pero solo para garfield
garfield.nombres = []

#creamos un nuevo gato
silvestre = Gato('Silvestre')
```

```
#si ahora mostramos todos los gatos (la Clase) y a garfield...

print(Gato.nombres) # ['Garfield','Bigotes','Silvestre']
# vemos ahora cómo bigotes se ve afectado
print(bigotes.nombres) # ['Garfield','Bigotes','Silvestre']
# pero garfield NO
print(garfield.nombres) # []
```

6. Privacidad

En Python, la privacidad en el código se gestiona principalmente a través de convenciones de nomenclatura y prácticas de diseño más que mediante mecanismos del lenguaje como puede ocurrir en lenguajes como Java (modificadores de acceso como private o protected)

- Un solo guion bajo (_nombre) como prefijo significa que ese atributo o método debe considerarse protegido para la clase, es decir, es interno y no debería usarse fuera de la misma.
- Doble guión bajo (__nombre) como prefijo significa que el atributo o método es privado y se establece “obligatoriamente” que no se utilice fuera del ámbito de la clase. Para este tipo de nombre se activa lo que se conoce en Python como *name mangling* que consiste en que se añade el nombre de la clase para que así sea más difícil que se produzcan colisiones con otros métodos o atributos de la misma clase o de cualquier otra que herede de la misma.

Ejemplo:

```
class Ejemplo:

    #atributos de clase

    _valor_protegido = 5
    __valor_privado = 10

    def __init__(self):
        self.publico = "Soy público"
        self._protegido = "Soy protegido (convención)"
        self.__privado = "Soy privado (name mangling)"

    def mostrar_privado(self):
        return self.__privado

obj = Ejemplo()
print(obj.publico)  # Accesible
print(obj._protegido)  # Accesible pero no recomendado
```

```
#print(obj.__privado)  # Error: AttributeError
print(obj.mostrar_privado()) # Acceso mediante método
print(obj._Ejemplo__privado)  # Acceso mediante name mangling
print(Ejemplo._Ejemplo__valor_privado) #En absoluto recomendado acceder así
```

7. Construcción de clases personalizadas

Construir clases personalizadas en Python te permite crear tipos de datos específicos que encapsulan atributos y comportamientos. Cuando se crea una instancia en Python se utilizan dos métodos: __**new**__ e __**init**__**.** Uno sirve como constructor de la instancia (__new__) y el otro como inicializador (__init__). Hay que tener muy en cuenta que en la mayoría de los casos, **no necesitas sobrescribir** __**new**__ porque Python ya proporciona una implementación predeterminada se encarga de crear la instancia de la clase. Pero sí será necesario cuando necesitas personalizar **cómo se crea el objeto** o incluso devolver un tipo diferente. Sobrescribir __**init**__ es común dado que lo utilizaremos para inicializar atributos o ejecutar lógica específica una vez que el objeto ha sido creado.

- **def __new__(cls, *args, **kwargs)**: se llama para crear una nueva instancia de la clase. Está presente en todas las clases de Python. Los argumentos son los usados para la creación de la clase en el constructor. Devuelve una instancia de un objeto que normalmente es la instancia de la clase **cls** que estamos tratando, pero no es obligatorio. Normalmente se utiliza **super().__new__(cls, *args, **kwargs)** para a continuación modificar la instancia creada. Si no se devuelve la instancia (return) el método __init__ no será invocado.
  Se debe sobrescribir en casos especiales donde controlas la creación del objeto

- **def __init__(self, *args, **kwargs):** se llama a continuación de __new__ y sirve para inicializar la instancia que retorna __new__. No necesita devolver ningún valor o puede devolver **None,** en otro caso eleva un error **TypeError**. Normalmente utiliza **super().__init__(*args, **kwargs)** o si se quiere indicar la clase se haría **super(NombreClase, self).__init__(*args, **kwargs)** de manera que se inicializa la instancia de la clase actual y los constructores de las clases superiores.

Veamos todo esto con un ejemplo.

Imagina que queremos crear una clase que dé de alta a empleados asignándoles un identificador de empleado que debe ser único. Si se da de baja un empleado, ese identificador puede reutilizarse.

Revisa el código y trata de entenderlo…

```
class Identificador:
    _ids_existentes = set()  # Variable de clase para llevar registro de los IDs
existentes

    def __new__(cls, id_empleado, *args, **kwargs):
        # Verificar si el ID ya existe
        if id_empleado in cls._ids_existentes:
            raise ValueError(f"El ID '{id_empleado}' ya está en uso. Usa otro.")
        print(f"Creando una nueva instancia con ID: {id_empleado}")
        instance = super().__new__(cls)  # Crear la instancia
        return instance

    def __init__(self, id_empleado, nombre):
        print(f"Inicializando instancia con ID: {id_empleado}")
        self.id_empleado = id_empleado
        self.nombre = nombre
        # Registrar el ID en la clase
        self._ids_existentes.add(id_empleado)

    def __del__(self):
        # Al eliminar una instancia, liberar su ID
        #print(f"Eliminando instancia con ID: {self.id_empleado}")
        self._ids_existentes.remove(self.id_empleado)


# Pruebas
try:
    obj1 = Identificador(1, "Antonio García")
    obj2 = Identificador(2, "Marta Alonso")
    obj3 = Identificador(1, "Pedro Osinaga")  # Esto generará un error
except ValueError as e:
    print(e)
except Exception as e:
    print(e)

# Liberar un ID y usarlo nuevamente
del obj1
obj4 = Identificador(1, "Beltrán Cortés")

# Verificar contenido de los objetos
print(f'El empleado "{obj2.nombre}" tiene el Id "{obj2.id_empleado}"')
print(f'El empleado "{obj4.nombre}" tiene el Id "{obj4.id_empleado}"')
```

Salida:

```
Inicializando instancia con ID: 1
Creando una nueva instancia con ID: 2
Inicializando instancia con ID: 2
El ID '1' ya está en uso. Usa otro.
Creando una nueva instancia con ID: 1
Inicializando instancia con ID: 1
El empleado "Marta Alonso" tiene el Id "2"
El empleado "Beltrán Cortés" tiene el Id "1"
```



¿Por qué necesitamos realmente utilizar __new__ y no solo __init__?

Podríamos utilizar la lógica que hemos creado en __new__ dentro de __init__ pero fíjate que si lo hiciésemos así, cuando comprobamos si el identificador de empleado ya lo tenemos asignado, la instancia estaría creada (es lo que hace __new__) de manera que estaríamos gastando recursos de memoria sin necesidad.

Veamos otro ejemplo: **corredor.py**

```
class Corredor:
    #declaramos atributo de clase privado
    __numeros_usados = set()

    # definimos __new__

    def __new__(cls, nombre, num=1):
        #comprobamos si el num ya está usado...
        while num in cls.__numeros_usados:
            num += 1

        # cuando no esté, lo añadimos al conjunto
        cls.__numeros_usados.add(num)
        # aseguramos que el constructor de la superclase Corredor se ejecuta
        instancia = super(Corredor, cls).__new__(cls)
        # y especificamos el atributo num con el valor actual
        instancia.num = num
        #por último no podemos olvidar devolver la instancia para que
        #se ejecute __init__
        return instancia

    # definimos __init__

    def __init__(self, nombre):

        # como ya tendremos en la instancia creada el atributo
```

```
        # num asignado, sólo nos queda asignar el valor del nombre
        self.nombre = nombre

    # definimos una función para eliminar un numero
    def __del__(self):
        self.__numeros_usados.remove(self.num)


# ejecutamos las funciones
juan = Corredor('Juan')
print(juan.nombre)
print(juan.num)
maria = Corredor('Maria')
print(maria.nombre)
print(maria.num)
marta=Corredor('Marta')
print(marta.nombre)
print(marta.num)
del maria
#observa como Silvia toma el número que deja Maria
silvia = Corredor('Silvia')
print(silvia.nombre)
print(silvia.num)
```

Salida:

```
Juan
1
Maria
2
Marta
3
Silvia
2
```

8. Propiedades en Clases

En ocasiones es necesario controlar los atributos y definir cómo se deben acceder, actualizar y qué hacer cuando se quieren eliminar. Para esto Python cuenta con el concepto de **propiedad (property).** Es común que cuando se tienen propiedades de clase, se definan las funciones para el tratamiento de esas propiedades. Lo que se conoce como getters, setters y deleters, es decir, cómo se obtiene, se inicializa y/o se elimina, además de un texto de documentación. Para ello, Python cuenta con la clase base **property,** cuyo constructor recibe los siguientes parámetros:

property(fget=None, fset=None, fdel=None, doc=None)

- fget(self): la función que se ejecuta al obtener el atributo (getter).

- fset(self): la función que se ejecuta al establecer el atributo (setter).
- fdel(self): la función que se ejecuta al eliminar el atributo (deleter).
- doc: una cadena de texto opcional para documentar la propiedad.

Veamos un ejemplo sencillo:

```
class Ejemplo:
    def __init__(self, valor):
        self._valor = valor

    # Función getter
    def get_valor(self):
        return self._valor

    # Función setter
    def set_valor(self, nuevo_valor):
        if nuevo_valor < 0:
            raise ValueError("El valor no puede ser negativo")
        self._valor = nuevo_valor

    def del_valor(self):
        del self._valor

    # Crear el objeto property
    valor = property(fget=get_valor, fset=set_valor, fdel=del_valor, doc="Soy la
propiedad 'valor'")

# Uso
obj = Ejemplo(10)
print(obj.valor)  # 10
obj.valor = 20
print(obj.valor)  # 20
print(Ejemplo.valor.__doc__)
obj.valor=-5
```

Como puedes comprobar, estamos usando property para crear el objeto **valor.** Este se comportará como un atributo controlado de la instancia que se crea de la clase. En nuestro ejemplo, creamos la instancia **obj** que tiene el atributo (instancia de property) **valor** mediante el que podemos acceder, establecer y/o eliminar el atributo.

Además, Python proporciona la posibilidad de utilizar **@property** como decorador definido en la clase de manera que, siendo el funcionamiento el mismo, permite que el uso sea más compacto y legible. Esto ocurre porque el decorador @property crea una instancia de la clase **property** automáticamente y asocia las funciones fget, fset y fdel.

La sintaxis es la siguiente:

- @property: define el acceso al atributo y define la cadena que se usará como documentación. Si sólo se utiliza este decorador, se crea una versión de sólo lectura del atributo, es decir, no se podrá establecer (setter) ni eliminar (deleter)
- @<atributo>.setter: define cómo se actualiza el atributo. Si no se define, no se puede modificar el atributo.
- @<atributo>.deleter: define cómo se elimina el atributo. Si no se define, no se puede eliminar el atributo.

Veamos el siguiente ejemplo:

```
class Ejemplo:
    def __init__(self, valor):
        self._valor = valor

    @property
    def valor(self):
        return self._valor

    @valor.setter
    def valor(self, nuevo_valor):
        if nuevo_valor < 0:
            raise ValueError("El valor no puede ser negativo")
        self._valor = nuevo_valor

# Uso
obj = Ejemplo(10)
print(obj.valor)  # 10
obj.valor = 20
print(obj.valor)  # 20
del obj.valor # AttributeError: can't delete attribute
```

Observa que en el ejemplo no se ha definido el decorador @valor.deleter, de manera que el atributo **valor** no se puede eliminar. Se definen, eso sí, los decoradores @property y @valor.setter para acceder actualizar el atributo **valor.** Observa que en ningún caso se ha realizado la llamada a **property** porque el decorador @property ha creado la instancia de la clase **property** automáticamente.

Un ejemplo más. *Se pretende crear una clase **Finalista** que guarde la clasificación de corredores que terminaron la carrera respetando el orden en que pasaron por la línea de meta. La clase **Corredor** es la misma que hemos definido anteriormente y es el único objeto*

*se usará como parámetro en el constructor de la clase **Finalista**. La clase **Finalista** deberá guardar el contador actual de todos los finalistas y cada finalista no puede modificar su posición.*

Para este ejemplo, vamos a crear la clase **Finalista** que importará la clase **Corredor** creada en el fichero **corredor.py** con anterioridad.

```
#importamos la clase Corredor del módulo corredor
from corredor import Corredor

# Creamos la clase Finalista
class Finalista:
    __finalizados = 0
    def __new__(cls,corredor):
        cls.__finalizados += 1
        puesto = cls.__finalizados
        instancia = super(Finalista, cls).__new__(cls)
        instancia._posicion = puesto
        return instancia
    def __init__(self,corredor):
        self.corredor = corredor

    @property
    def posicion(self):

        """Esta será la posición final del corredor"""
        return self._posicion

#Probamos el código, creando primero los corredores
juan = Corredor('Juan')
ana = Corredor('Ana')
#creamos los finalistas
juan_finalista = Finalista(juan)
ana_finalista = Finalista(ana)
#mostramos la posición de cada uno
print(ana_finalista.posicion) # 2
print(juan_finalista.posicion) # 1
#Intentamos modificar la posición de Juan
juan_finalista.posicion = 3 # AttributeError: can't set attribute
```

Observa que el número de la posición de los corredores en meta (Finalista) se calcula en __**new**__ mediante un atributo de clase protegido con el doble '__' de forma secuencial asignado automáticamente a cada corredor. Si se intenta modificar la posición de un corredor no se permitirá dado que se ha definido el decorador @**property** pero sólo con getter y no con setter.

9. Métodos

En Python, los métodos son funciones que están asociadas a un objeto y pueden operar sobre los datos de ese objeto. Se definen dentro de la clase y pueden acceder a los atributos. Ya hemos visto métodos como __new__ o __init__. Ahora vamos a ver los tipos de métodos, sus características y cómo crear métodos personalizados para clases e instancias.

Existen principalmente tres tipos de métodos: de instancia, de clase y estáticos.

9.2. Métodos de instancia.

Los más comunes. Operan sobre las instancia de la clase y acceden a sus atributos mediante **self**. La forma de construirlos es exactamente igual que construir funciones con la salvedad de que el primer parámetro hace referencia a la instancia asociada al método.

El siguiente ejemplo define la clase Persona con dos métodos de instancia, el método **__init__** que ya conocemos y el método **saludar** que recupera los valores de los atributos del objeto:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        return f"Hola, mi nombre es {self.nombre} y tengo {self.edad} años."

# Crear un objeto
persona1 = Persona("Ana", 30)

# Llamar al método
print(persona1.saludar())  # Salida: Hola, mi nombre es Ana y tengo 30 años.
```

Otro ejemplo:

```
class Coche:
    def __init__(self, marca, modelo, año):
        self.marca = marca
        self.modelo = modelo
        self.año = año
        self.kilometraje = 0

    # Método de instancia que devuelve una descripción del coche
    def describir_coche(self):
        return f"{self.año} {self.marca} {self.modelo} - {self.kilometraje} km"
```

```
    # Otro método de instancia que modifica los kilómetros recorridos
    def actualizar_kilometraje(self, nuevos_kilometros):
        if nuevos_kilometros >= self.kilometraje:
            self.kilometraje = nuevos_kilometros
        else:
            print("¡No se puede reducir el kilometraje!")

# Crear una instancia
mi_coche = Coche("Mazda", "CX-5", 2017)

# Llamar a métodos de instancia
print(mi_coche.describir_coche())  # 2017 Mazda CX-5 - 0 km

mi_coche.actualizar_kilometraje(15000)
print(mi_coche.describir_coche())  # 2017 Mazda CX-5 - 15000 km
```

9.3. Métodos de clase.

Afectan a la clase en general y no a la instancia en particular. Se utiliza el decorador **@classmethod** en su definición. El primero parámetro, por convención, es **cls** referenciando a la propia clase (class no se puede utilizar al ser una palabra reservada)

Ejemplo:

```
class Persona:
    contador_personas = 0  # Atributo de clase

    def __init__(self, nombre):
        self.nombre = nombre
        Persona.contador_personas += 1

    # Método de clase
    @classmethod
    def total_personas(cls):
        return f"Se han creado {cls.contador_personas} personas."

# Crear objetos
p1 = Persona("Ana")
p2 = Persona("Juan")

# Llamar al método de clase
print(Persona.total_personas())  # Salida: Se han creado 2 personas.
```

Como puedes comprobar, al llamar al método de clase se referencia con la propia clase **Persona.**

Los métodos de clase tienen un uso particular cuando se quieren hacer instancias de una clase que se crean de forma diferente al constructor ordinario como por ejemplo cuando necesitamos inicializar con diferentes formatos, inicialización con diferentes bases de datos o inicialización usando diferentes parámetros con una lógica particular, por nombrar algunos ejemplos.

Veamos el siguiente ejemplo en el que se crean animales pero de distinta forma, de manera que usamos la inicialización estándar (__init__), usando una cadena de caracteres separada por comas o de forma predefinida para cada animal:

Ejemplo:

```
class Animal:
    def __init__(self, tipo, volumen, masa):
        self.tipo = tipo
        self.volumen = volumen
        self.masa = masa

    @classmethod
    def desde_str(cls, cadena):
        tipo, volumen, masa = cadena.split(',')
        return cls(tipo, float(volumen), float(masa))

    @classmethod
    def gato(cls):
        return cls('Gato', 120, 3.8)

    @classmethod
    def perro(cls):
        return cls('Perro', 500, 25.4)

cebra = Animal('Cebra',15000,150)
elefante = Animal.desde_str('Elefante, 300000, 2600')
gato = Animal.gato()
perro = Animal.perro()
#Mostrar los atributos de los objetos
print(cebra.tipo, cebra.volumen, cebra.masa)
print(elefante.tipo, elefante.volumen, elefante.masa)
print(gato.tipo, gato.volumen, gato.masa)
print(perro.tipo, perro.volumen, perro.masa)
#Mostrar el tipo de los objetos
print(type(cebra),type(elefante),type(gato),type(perro))
```

Si ejecutas y observas el resultado del ejemplo anterior, podrás comprobar que se han creado distintos animales de forma distinta pero el tipo para todos y cada uno de ellos es la clase “**Animal**”.

9.4. Métodos estáticos.

No dependen ni de la clase (cls) ni de la instancia (self). Son funciones dentro de la clase que no acceden a atributos ni de la clase ni del objeto.

Se utilizan para unir código relacionado con una clase en particular, que podría estar definido de forma independiente a nivel de módulo, pero que se tiene en la clase para hacerlo más legible o porque sencillamente, fuera de la clase no tiene sentido.

Es necesario hacer uso del decorador **@staticmethod.**

Tomando la clase anterior "Animal", veamos cómo podemos utilizar un método estático para definir un método estático:

```
class Animal:

    ...

    def peso(self):
      return self.masa * self.gravedad()

    @staticmethod
    def gravedad():
      return 9.8

...
print(Animal.gravedad()) # 9.8
print(elefante.peso()) # 25480.0
```

10. Métodos mágicos.

Permiten definir el comportamiento de los objetos de manera personalizada. Controlan el comportamiento de las clases. Hacen que los objetos se comporten como tipos nativos. Permiten la sobrecarga de operadores y consiguen que el código sea más legible y Pythonic.

Se crean para definir protocolos comunes a todas las clases y tipos de datos, tanto del núcleo de Python como creadas individualmente.

En Python, las clases que pretenden utilizar determinadas operaciones necesitarán desarrollar métodos especiales para conseguirlo.

Veámoslo con este ejemplo. Creamos una clase Planta en la que se guarda el nombre de la planta, el tipo y la altura. Para poder comparar (igual,mayor, mayor o igual, menor, menor o igual) una instancia de Planta con otra, debemos definir en la clase un método o los métodos especiales (mágicos) que no colisionen fácilmente con otros métodos y que estén conectados con los operadores por defecto, de manera que creamos lo siguiente:

__eq__ (==) para comprobar si dos plantas son del mismo tipo

__lt__ (<) para determinar si una planta es más pequeña que otra en altura

__le__ (<=) para determinar si una planta es más pequeña o igual que otra en altura

__gt__ (>) para determinar si una planta es mayor que otra en altura

__ge__ (>=)para determinar si una planta es mayor o igual que otra en altura

__str__ para representar la planta de forma legible

Veamos el código:

```
class Planta:
    """Representa una planta con un nombre y una altura."""
    def __init__(self, nombre, tipo, altura):
        self.nombre = nombre
        self.tipo = tipo
        self.altura = altura  # Altura en centímetros

    def __eq__(self, otra_planta):
        """Comprueba si dos plantas tienen la misma altura."""
        return self.tipo == otra_planta.tipo

    def __lt__(self, otra_planta):
        """Comprueba si una planta es más baja que otra."""
        return self.altura < otra_planta.altura

    def __le__(self, otra_planta):
        """Comprueba si una planta es más baja o igual a otra."""
        return self.altura <= otra_planta.altura

    def __gt__(self, otra_planta):
        """Comprueba si una planta es más alta que otra."""
        return self.altura > otra_planta.altura

    def __ge__(self, otra_planta):
        """Comprueba si una planta es más alta o igual a otra."""
        return self.altura >= otra_planta.altura
```

```
    def __str__(self):
        """Representación de la planta."""
        return f"{self.nombre} ({self.altura} cm)"

# Ejemplo de uso
planta1 = Planta("Rosa", "rosa",30)
planta2 = Planta("Girasol", "girasol",150)
planta3 = Planta("Tulipán","tulipán", 30)

# Comparaciones
print(planta1 == planta3)  # False (Distinto tipo)
print(planta1 < planta2)   # True (Rosa es más baja que Girasol)
print(planta2 > planta1)   # True (Girasol es más alta que Rosa)
print(planta1 <= planta3)  # True (Misma altura)
print(planta2 >= planta3)  # True (Girasol es más alta)
print(planta1)  # Rosa (30 cm)
```

Como se puede observar, al definir todos los "métodos mágicos" podemos utilizar los operadores (==,<,<=,>,>=) para realizar las comparaciones entre plantas de forma sencilla.

Cabe destacar igualmente que para las operaciones análogas (por ejemplo, igualdad y desigualdad, mayor y menor…) basta con implementar una de las dos para tener soporte de ambas. Si compruebas en el ejemplo anterior la desigualdad **(!=)** que no se ha definido, verás que funciona dado que se ha definido la igualdad (__**eq**__)

Además, se ha creado el método mágico __**str**__ para permitir describir una planta de forma legible. (Prueba a comentar el método __**str**__ y observa cómo se muestra por pantalla "planta1" sin la acción del método.

El conjunto de métodos mágicos que se pueden implementar para construir una clase y permitir personalizar cómo se comportan los objetos serían los siguientes:

- def __new__(cls, *args, **kwargs): constructor de la clase que se encarga de generar instancias nuevas.
- def __init__(self, *args, **kwargs): inicializa la clase y es llamado después de __new__
- def __del__(self): se llama cuando se va a eliminar una instancia.
- def __str__(self): es llamado por las funciones str(), print() u format() y se utiliza para mostrar un aspecto "amigable" del objeto.
- def __bytes__(self): es usado por la función bytes() para devolver un tipo byte con la representación binaria de la instancia. Muy útil para "serializar" objetos.

Ejemplo:

```
class Persona:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def __bytes__(self):
        return f"{self.nombre},{self.edad}".encode("utf-8")  # Codificar en bytes

p = Persona("Ana", 25)
print(bytes(p))
# Salida: b'Ana,25'  (Ahora sí se puede convertir a bytes)
```

- def __format__(self, format_spec): es llamado al utilizar el método format(). El segundo parámetro describe las opciones de formateado.
- def __hash__(self): es utilizado por la función hash() y va a permitir que los objetos sean usados como claves en diccionarios o agregar objetos a conjuntos (set()) dado que éstos almacenan valores únicos. Si el objeto no define el método __eq__ no debe implementar tampoco este método.
- def __bool__(self): este método se utiliza para llamar a la función bool() y para comprobar la veracidad del objeto en expresiones. Si no se implementa el método __len__ ni este método, se considera True a la veracidad del objeto dado que es el valor por defecto.

Es muy importante tener claro que ninguno de estos métodos descritos es necesario para la creación de clases pero si son de gran utilidad y conviene tenerlos en cuenta.

11. Personalizar el acceso a los atributos.

Al definir clases en Python se pueden especificar cómo se accede a los atributos utilizando los siguientes métodos mágicos:

- def __getattribute__(self, nombre): se llama siempre que se quiere acceder al atributo de un objeto utilizando **nombre.**
- def __getattr(self, nombre): es llamado cuando el método anterior falla y eleva una excepción **AttributeError**
- def setattr(self, nombre, valor): se llama cuando se asigna un valor a un atributo de la instancia.
- def __delattr__(self, nombre): es llamado cuando se pretende eliminar un atributo de un objeto utilizando **del objeto.**
- del __dir__(self): se llama cuando se utiliza la función dir() y debe devolver un objeto tipo secuencia.

12. Polimorfismo (Duck Typing)

En POO existe un concepto denominado polimorfismo que se basa en que, objetos de diferentes tipos pueden tener características similares y por tanto, compartir nombres de métodos o atributos.

Un ejemplo muy simple es la definición de diferentes clases que tengan el mismo método, pero que devuelvan resultados diferentes.

Un ejemplo sencillo:

```
class Perro:
    def sonido(self):
        return "Guau Guau"

class Gato:
    def sonido(self):
        return "Miau Miau"

# Función que usa polimorfismo
def hacer_sonido(animal):
    print(animal.sonido())

# Probamos con diferentes clases
mascota1 = Perro()
mascota2 = Gato()

hacer_sonido(mascota1)  # Salida: Guau Guau
hacer_sonido(mascota2)  # Salida: Miau Miau
```

Como podemos comprobar en el ejemplo, la clase Perro y la clase Gato definen el mismo método y en cada caso hacen cosas distintas. La función **hacer_sonido(animal)** se encargará de ejecutar uno y otro código en función del objeto que se le pase por parámetro (Perro o Gato).

Cuando se hace uso de la herencia y varias clases comparten los mismos atributos de la clase base y necesitan redefinir métodos, se utiliza el polimorfismo.

Observa y analiza el siguiente ejemplo:

```
class Animal:  # Clase base
    def hacer_sonido(self):
        return "Algún sonido..."  # Método genérico (puede ser sobrescrito)

class Perro(Animal):  # Clase hija
    def hacer_sonido(self):
        sonido_padre=super().hacer_sonido()
        return f'{sonido_padre} "Guau Guau"'  # Sobrescribiendo el método
```

```
class Gato(Animal):  # Clase hija
    def hacer_sonido(self):
        return "Miau Miau"  # Sobrescribiendo el método

# Lista de animales (pueden ser de diferentes clases)
animales = [Perro(), Gato(), Animal()]

for animal in animales:
    print(animal.hacer_sonido())
```

salida:

Algún sonido... "Guau Guau"

Miau Miau

Algún sonido...

13. Herencia.

Es una de las herramientas más potentes de la POO. Permite que, partiendo de una clase cualquiera, tengamos una nueva clase que hereda los métodos y atributos de la primera, consiguiendo una clase más específica. La clase original recibe el nombre de “clase base” o “clase padre” y la nueva se denomina “clase hija” o “clase heredera”.

La herencia permite la **reutilización** de código, la **extensibilidad**, de manera que se agrega nuevo código a la clase original y el **polimorfismo** que permite que podamos redefinir métodos de la clase original. Para hacer que una clase hija herede de otra clase padre, la clase hija se define con el nombre de la clase padre entre paréntesis

class Hija(ClassPadre):

En Python, todas las clases heredan por defecto de la clase **object.**

Veamos un ejemplo:

```
class MiStr:

    def __init__(self, cadena):
        self.cadena = cadena

    def longitud(self):
        return len(self.cadena)

    def info(self):
        return f'La cadena "{self.cadena}" tiene {self.longitud()} caracteres'

class MiStrMayus(MiStr):
```

```
    def info(self):
        return self.cadena.upper()
class MiStrMinus(MiStr):
    def info(self):
        return self.cadena.lower()

m_str = MiStr('Hola Mundo')
print(m_str.longitud)
print(m_str.info())
m_str_mayus = MiStrMayus('Hola Mundo')
print(m_str_mayus.longitud())
print(m_str_mayus.info())
m_str_minus = MiStrMinus('Hola Mundo')
print(m_str_minus.longitud())
print(m_str_minus.info())
```

En el ejemplo anterior se puede ver que la función **longitud** sólo se define en la clase padre, pero es accesible desde cualquier clase hija. Esto es así porque Python, cuando no encuentra un método o atributo en la instancia en la que se encuentra, va recorriendo la jerarquía de las clases padre intentando encontrarlo. Si no lo encuentra devuelve un error del tipo **AttributeError.** Por ese mismo motivo, al encontrar el método **info** definido en cada una de las clases hijas, no ejecuta el de la clase padre. Esto es lo que se conoce como **sobrescritura de métodos.** Esta sobrescritura también puede ser de **atributos**.

Como curiosidad, podemos ver la jerarquía de las clase ejecutando el método __**mro**__

Ejecuta y observa las siguientes sentencias sobre el código del ejemplo anterior:

print(MiStr.__mro__)

print(MiStrMayus.__mro__)

print(MiSTRMinus.__mro__)

Con la herencia lo que se persigue es hacer una versión más específica de la clase padre, aprovechando al máximo el código. Si se quiere reutilizar el código de la clase padre, Python, al igual que otros lenguajes, proporciona un mecanismo para llamar a las funciones de la clase padre y obtener los resultados desde la instancia hija. Esto se hace mediante la función **super:**

super().__init__(*args, **kwargs): inicializa la clase hijo llamando al método __init__del padre. Los parámetros deben coincidir con los del padre.

super().mi_metodo(*args, **kwargs): sirve para extender la funcionalidad de un método definido en la clase padre haciendo uso del valor que devuelve.

Antes hemos visto el uso de **super** en el ejemplo de la clase Perro heredando de Animal.

Observa este ejemplo de Empleados y la creación de un Gerente que hereda de Empleado.

```
class Empleado:
    """Clase base para todos los empleados"""
    def __init__(self, nombre, salario):
        self.nombre = nombre
        self.salario = salario

    def calcular_pago(self):
        """Método que será sobrescrito en las subclases"""
        return self.salario

    def __str__(self):
        return f"{self.nombre} - Salario: {self.salario}€"

# Clase Gerente que hereda de Empleado
class Gerente(Empleado):
    def __init__(self, nombre, salario, bono):
        super().__init__(nombre, salario)  # Llamamos al constructor de Empleado
        self.bono = bono  # Nuevo atributo

    def calcular_pago(self):
        return super().calcular_pago() + self.bono  # Llamamos al método padre y
sumamos el bono

    def __str__(self):
        return super().__str__() + f" + Bono: {self.bono}€"
```

Como se puede observar, utilizamos **super()** para inicializar los objetos de la clase Gerente con el método de la clase padre Empleado.

De igual modo, el método **calcular_pago** de la clase Gerente extiende la funcionalidad de la clase Empleado mediante la llamada al método super().calcular_pago(). Lo mismo ocurre con __str__

Esto nos permite crear nuevas clases reutilizando código y de esa manera conseguimos que sea más organizado y fácil de mantener.

13.1. Herencia Múltiple.

Python es uno de los pocos lenguajes que permiten la herencia múltiple. La herencia múltiple permite que una clase tenga más de una clase padre, permitiendo tener funcionalidades de distintas clases en una sola.

Ejemplo de cómo funciona la herencia múltiple:

```
class Empleado:
    """Clase base para empleados"""
    def __init__(self, nombre, edad, salario_base):
        self.nombre = nombre
        self.edad = edad
        self.salario_base = salario_base

    def mostrar_info(self):
        return f"Empleado: {self.nombre}, Edad: {self.edad}, Salario Base:
{self.salario_base}€"


class Pagos:
    """Clase que maneja pagos y bonificaciones"""
    def calcular_pago(self, bonificacion):
        return self.salario_base + bonificacion


class Reportes:
    """Clase que genera reportes de empleados"""
    def generar_reporte(self):
        return f"Reporte: {self.nombre}, Edad: {self.edad}, Salario:
{self.salario_base}€"


class Gerente(Empleado, Pagos, Reportes):
    """Clase que hereda de Empleado, Pagos y Reportes"""
    def __init__(self, nombre, edad, salario_base, departamento):
        super().__init__(nombre, edad, salario_base)  # Llamamos al constructor de
Empleado
        self.departamento = departamento

    def mostrar_info(self):
        return super().mostrar_info() + f", Departamento: {self.departamento}"


#Crear un objeto de la clase Gerente
gerente = Gerente("Ana López", 40, 5000, "Tecnología")

#Usar métodos de diferentes clases
print(gerente.mostrar_info())  # Método de Empleado
print("Salario Total:", gerente.calcular_pago(1000))  # Método de Pagos
print(gerente.generar_reporte())  # Método de Reportes
```

La clase Empleado define los atributos básicos del empleado (nombre, edad, salario_base).

La clase Pagos nos permite calcular el pago total (calcular_pago(bonificacion)) de manera que se añadirá al salario_base la bonificación .

La clase Reportes genera un informe con los datos de, en este caso, el empleado (generar_reporte()).

La clase Gerente hereda de las tres clases -mutiherencia- (Empleado, Pagos, Reportes) y combina todas sus funcionalidades.

Observa que, si creamos una instancia directamente de la clase Reportes e intentamos llamar al método generar_reporte() no mostrará nada porque la clase en sí misma no tiene esos atributos (nombre, edad, salario_base). De hecho, elevará una excepción **AttributeError.** Sólo al definir la instancia de Gerente podrán utilizarse todas las funcionalidades de las demás clases de las que hereda.

Cuando una clase hereda de múltiples padres, Python sigue un orden de búsqueda de métodos llamado **MRO (Orden de resolución del método),** que normalmente viene definido por el orden en el que se define la clase.

Ejecuta **print(Gerente.mro())** y prueba que el orden en que se buscarán los métodos vendrá determinado por Gerente, Empleado, Pagos, Reportes, Object tal y como se ha definido la clase (`class Gerente(Empleado, Pagos, Reportes))`

En la herencia múltiple, el uso del método **super()** cobra importancia en el sentido de que se aplica el **MRO** en su ejecución.

En nuestro ejemplo, la sentencia **return super().mostrar_info()** de la función mostrar_info(self) de la clase Gerente, buscará por orden en la clase Empleado y como lo encuentra, ejecutará ese método.´

Se recomienda hacer un uso moderado de la herencia múltiple.

13.2. La clases MIXIN

Para cuando tienes necesidad de agregar funcionalidad común a múltiples clases, evitando código duplicado de manera que consigas una estructura de herencia más flexible, Python dispone de las clases **Mixin.**

Una clase **Mixin** en una clase que no se puede instanciar sola y que se usa sólo para proporcionar métodos a otras clases. No define atributos de instancia (__init__()), debe combinarse con otras clases mediante herencia múltiple y es independiente de la jerarquía de clases principal.

Las clases **Mixin** son muy utilizadas en los ORM (mapeo de objeto relacional) dado que permiten agregar funcionalidades reutilizables a los modelos de bases de datos sin necesidad de repetir código.

Su utilización en los ORM es sólo un ejemplo porque existen clases **Mixin** que se utilizan en diferentes frameworks como Django o Flask e incluso en contextos como APIs o juegos.

En Django, por ejemplo, existen multitud de clases **Mixin** que pretenden ayudar al desarrollador tales como *DetailView, SingleObjectMixin, TemplateResponseMixin…*

Veamos un ejemplo bastante sencillo de cuando y de qué manera utilizar las clases **Mixin**

Imagina que tenemos dos clases, Producto y Usuario (no vamos a hacer nada con ellos) que guardan información, en el caso del producto su nombre y su precio y en el caso del usuario, su nombre y su email. En cada una de estas clases lo único que tenemos como base es un método de inicialización y un método que muestra la información de cada objeto.

Este sería el ejemplo:

```
class Producto():
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def __str__(self):
        return f"Producto: {self.nombre}, Precio: {self.precio}"

# Clase Usuario que usa AuditMixin
class Usuario():
    def __init__(self, nombre, email):
        self.nombre = nombre
        self.email = email

    def __str__(self):
        return f"Usuario: {self.nombre}, Email: {self.email}"

# Crear un producto
producto = Producto(nombre="Laptop", precio=1200.00)
print(producto)  # Muestra los detalles del producto

# Crear un usuario
usuario = Usuario(nombre="Juan Pérez", email="juan@example.com")
print(usuario)  # Muestra los detalles del usuario
```

Ahora suponemos que queremos dotar de más funcionalidad a las dos clases de manera que podamos guardar la fecha de creación de un objeto de alguna de estas clases e imprimir un mensaje cada vez que se crea el objeto.

Para ello vamos a crear las clases Mixin siguientes:

```
from datetime import datetime

# Mixin para agregar auditoría (fecha de creación)
class AuditMixin:
    def __init__(self, *args, **kwargs):
        self.created_at = datetime.now()

# Mixin para agregar logging (imprimir mensajes)
class LoggingMixin:
    def __init__(self, *args, **kwargs):
        print(f"{self.__class__.__name__} creado en {datetime.now()}")
```

Ahora lo que debemos hacer es reescribir el código de las clases Producto y Usuario para que hereden de las clases Mixin:

```
# Clase Producto que usa AuditMixin y LoggingMixin
class Producto(AuditMixin, LoggingMixin):
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio
        super().__init__()  # Llama a los mixins

    def __str__(self):
        return f"Producto: {self.nombre}, Precio: {self.precio}, Creado:
        {self.created_at}"

# Clase Usuario que usa AuditMixin
class Usuario(AuditMixin):
    def __init__(self, nombre, email):
        self.nombre = nombre
        self.email = email
        super().__init__()  # Llama al mixin

    def __str__(self):
        return f"Usuario: {self.nombre}, Email: {self.email}, Creado:
        {self.created_at}"
```

Si observas el código, en la inicialización (creación) de cada objeto se llama a los Mixins. En el caso de Producto, como hereda de las dos clases, se llama a los dos método __init__. En el caso de Usuario, sólo hereda de AuditMixin, por tanto, ejecutará el __init__de dicha clase.

Prueba el código completo añadiendo las sentencias de creación de los objetos:

```
# Crear un producto
producto = Producto(nombre="Laptop", precio=1200.00)
print(producto)  # Muestra los detalles del producto

# Crear un usuario
usuario = Usuario(nombre="Juan Pérez", email="juan@example.com")
print(usuario)  # Muestra los detalles del usuario
```

14. Conclusión.

El uso de clases permite manejar objetos con estado y comportamiento, estructurando y reutilizando código. La programación orientada a objetos nos ayudan a modelar entidades del mundo real de manera clara. Las ventajas de usar clases son: **encapsulación,** agrupando datos (atributos) y comportamientos (métodos) en un solo lugar; reutilización de código gracias a la **herencia; abstracción,** ocultando detalles de implementación y exponiendo sólo lo necesario; **polimorfismo,** que permite que objetos diferentes respondan al mismo método de manera distinta.